SR UNIVERSITY

## School of Computer Science and Artificial Intelligence

# Lab Assignment # 11.3

| | |
|---|---|
| **Program** | **: B. Tech (CSE)** |
| **Specialization** | **:** |
| **Course Title** | **: AI Assisted coding** |
| **Course Code** | **:** |
| **Semester** | **: II** |
| **Academic Session** | **: 2025-2026** |
| **Name of Student** | **: V. Rikith** |
| **Enrollment No.** | **: 2403A51L35** |
| **Batch No.** | **: 52** |
| **Date** | **:24-02-2026** |

## Task 1: Smart Contact Manager (Arrays & Linked Lists)

### AI Prompt Used:

Generate Python code for a Contact Manager using arrays and linked lists with add, search, and delete operations.

### Array Implementation Explanation:

In this implementation, Python lists are used as arrays to store contact dictionaries. Insertion is efficient at the end (O(1)), but search and deletion require traversal (O(n)).

### Array Implementation Code:

task1.py > ContactManagerLinkedList > delete_contact

```python
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print("Contact Added Successfully")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                return contact
        return "Contact Not Found"

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                self.contacts.remove(contact)
                return "Contact Deleted"
        return "Contact Not Found"



cm = ContactManagerArray()
cm.add_contact("Rahul", "9876543210")
cm.add_contact("Anjali", "9123456789")

print(cm.search_contact("Rahul"))
print(cm.delete_contact("Rahul"))
```

### Linked List Explanation:

Linked List uses dynamic memory allocation. Each node stores contact details and a reference to the next node. Insertion at the beginning is O(1). Search and delete require traversal O(n).

### Linked List Code:

```python
class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact Added Successfully")

    def search_contact(self, name):
        temp = self.head
        while temp:
            if temp.name == name:
                return {"name": temp.name, "phone": temp.phone}
            temp = temp.next
        return "Contact Not Found"

    def delete_contact(self, name):
        temp = self.head
        prev = None

        while temp:
            if temp.name == name:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                return "Contact Deleted"
```

**Sample Output:**

```
PS D:\python> python task1.py
Contact Added Successfully
Contact Added Successfully
{'name': 'Rahul', 'phone': '9876543210'}
Contact Deleted
Contact Not Found
PS D:\python>
```

SR UNIVERSITY

## Task 2: Library Book Search System (Queue & Priority Queue)

### AI Prompt Used:

Generate enqueue and dequeue methods for Queue and Priority Queue in Python.

### Queue Explanation:

Queue follows FIFO (First In First Out). Requests are handled in the order received.

### Queue Code:

```
task2.py > ...
from collections import deque

class LibraryQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if self.queue:
            return self.queue.popleft()
        return "No Requests"


lq = LibraryQueue()
lq.enqueue("Student - Book A")
lq.enqueue("Student - Book B")

print(lq.dequeue())
print(lq.dequeue())
```

### Priority Queue Explanation:

Priority Queue assigns higher priority to faculty requests using heapq. Faculty requests are processed before student requests **Sample Output:**

```
PS D:\python> python task2.py
Student - Book A
Student - Book B
PS D:\python>
```

## Task 3: Emergency Help Desk (Stack)

### AI Prompt Used:

Generate stack operations push, pop, peek, is_empty in Python.

### Explanation:

Stack follows LIFO (Last In First Out). The most recent ticket is resolved first.

### Stack Code:

```
task3.py > ...
class HelpDeskStack:
    def __init__(self):
        self.stack = []
        self.max_size = 10

    def push(self, ticket):
        if len(self.stack) < self.max_size:
            self.stack.append(ticket)
        else:
            print("Stack Overflow")

    def pop(self):
        if self.stack:
            return self.stack.pop()
        return "Stack Underflow"

    def peek(self):
        if self.stack:
            return self.stack[-1]
        return "Stack Empty"

    def is_empty(self):
        return len(self.stack) == 0

    def is_full(self):
        return len(self.stack) == self.max_size
```

### Sample Output:

```
PS D:\python> python task3.py
Email Not Working
Printer Error
PS D:\python>
```

## Task 4: Hash Table with Chaining

### AI Prompt Used:

Generate a HashTable class in Python with insert, search, delete and collision handling using chaining.

### Explanation:

Hash Table uses a hash function to map keys to indices. Collisions are handled using chaining (list of key-value pairs at each index).

### Hash Table Code:

task4.py > HashTable > delete

```python
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
        self.table[index].append([key, value])

    def search(self, key):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return "Key Not Found"

    def delete(self, key):
        index = self.hash_function(key)
        for i, pair in enumerate(self.table[index]):
            if pair[0] == key:
                self.table[index].pop(i)
                return "Key Deleted"
        return "Key Not Found"
```

### Sample Output:

```
PS D:\python> python task4.py
Inserted key 'apple' at index 1
Inserted key 'banana' at index 1
Inserted key 'melon' at index 3
Inserted key 'grape' at index 1
[[], [['apple', 100], ['banana', 200], ['grape', 500]], [], [['melon', 400]], [['orange', 300]]]
Search for banana: 200
Search for cherry: Key Not Found
Deleted key 'orange' from index 4
Key Deleted
Key 'cherry' not found to delete
Key Not Found
[[], [['apple', 100], ['banana', 200], ['grape', 500]], [], [['melon', 400]], []]
PS D:\python>
```

## Task 5: Real-Time Application Challenge

### Mapping Table:

Attendance Tracking → Hash Table (Fast search)
Event Registration → Linked List (Dynamic size)
Library Borrowing → Queue (FIFO)
Bus Scheduling → Graph (Routes mapping)
Cafeteria Orders → Queue (Sequential processing)

### Selected Feature Implementation (Attendance System):

```
task5.py > ...
class At
    def
        (method) def mark_attendance(
            self: Self@AttendanceSystem,
            student_id: Any,
            status: Any
        ) -> None
    def mark_attendance(self, student_id, status):
        self.records[student_id] = status

    def check_attendance(self, student_id):
        return self.records.get(student_id, "No Record Found")


attendance = AttendanceSystem()
attendance.mark_attendance("23CS101", "Present")
attendance.mark_attendance("23CS102", "Absent")

print(attendance.check_attendance("23CS101"))
```

**Sample Output**

```
PS D:\python> python task5.py
Present
PS D:\python>
```